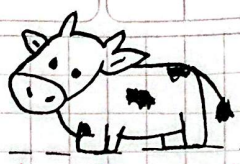
DD MM AA



Polymorphism, Interfaces, and Abstraction

Three related concepts with different roles



1. Polymorphism = Many Forms

- One common reference type; different object behaviors
- For overridden instance methods, the actual object determines which implementation runs at runtime
- Example: sound() returns Cluck for a Chicken and Moo for a Cow
- Benefit: new implementation can fit existing client code

2. Interface = a contract

- A Java type that specifies operations and object must support
- A class uses implements; an interface extends another interface
- A class can implement multiple interfaces
- An interface cannot be instantiated directly
- Interfaces can also provide default and static methods

3. Abstraction = Focus on Essentials

- Model relevant features and hide unnecessary implementation details
- Example: a farm system models an animal's ID and behavior, but leaves out details unrelated to that system.
- Abstraction is a design principle
- Interfaces, abstract classes, and ordinary classes can express it.

One Java example connects them

```
interface FarmAnimal {
    String sound();
}
class Chicken implements FarmAnimal {
    @Override
    public String sound() {return "Cluck";}
}
class Cow implements FarmAnimal {
    @Override
    public String sound() {return "Moo";}
}
```

Inside main():

```
FarmAnimal a = new Chicken();
FarmAnimal b = new Cow();
System.out.println(a.sound()); // Cluck
System.out.println(b.sound()); // Moo
```

- Interface: FarmAnimal defines the sound() contract
- Abstraction: callers use sound() without its implementation details
- Polymorphism: the same call uses Chicken's or Cow's implementation.

Interface vs Abstract Class

Feature	Interface	Abstract class
Main role	Contract / capability	Shared base for subclasses
Instance fields	No	Allowed
Constructors	No	Allowed
Create directly	No	No

- A concrete class must implement all inherited abstract methods
- Java classes extend one class and may implement several interfaces

Overriding vs Overloading

- Overriding: replace an inherited instance-method implementation; the object's class determines behavior at run time.
- Overloading: same name, different parameter lists; the compiler selects the method signature